Results in Engineering 27 (2025) 106263

Contents lists available at ScienceDirect

Results in Engineering

journal homepage: www.sciencedirect.com/journal/results-in-engineering

Research paper

Deep learning and machine learning models for portfolio optimization: Enhancing return prediction with stock clustering

Mahdi Ashrafzadeh [a], Mohammad Sadrani [b,*], Sarfaraz Hashemkhani Zolfani [c]

[a] Department of Industrial Engineering, Amirkabir University of Technology, Tehran, Iran
[b] "Friedrich List" Faculty of Transport and Traffic Sciences, TU Dresden, Dresden 01069, Germany
[c] School of Engineering, Universidad Catolica del Norte, Larrondo 1281, Coquimbo, Chile

ARTICLE INFO

Keywords:
Portfolio optimization
Deep learning
Clustering
Return prediction
Mean-variance with forecasting

ABSTRACT

For stock market investors, precise stock forecasts lead to improved decisions and greater profits. This study focuses on advancing stock return predictions through a clustering approach applied to selected stocks. Deep learning (DL) and Machine Learning (ML) models are trained on representative data from each cluster to forecast stock returns within the clusters. We explore the combination of three clustering methods [Dynamic Time Warping (DTW), K-Means, and Density-Based Spatial Clustering of Applications with Noise (DBSCAN)] with four prediction models [Long Short-Term Memory (LSTM), Convolutional Neural Network (CNN), Random Forest (RF), and eXtreme Gradient Boosting (XGBoost)]. The effectiveness of each DL and ML model combined with clustering is tested using the Diebold-Mariano (DM) test, confirming their superiority over methods lacking clustering. We also introduce a two-step process for portfolio optimization using predicted returns. High-predicted return stocks are initially preselected, followed by applying the Mean-Variance with Forecasting (MVF) optimization model to allocate stocks optimally. Using data from the New York Stock Exchange's large-cap stocks (2013 to 2022), results demonstrate the superiority of each clustering-based prediction model combined with MVF. The findings are further validated through daily trading simulations on test data from 2017 to 2022.

1. Introduction

In the realm of stock market investments, making precise stock forecasts is critical for informed decision-making and maximizing profits [1–3]. The introduction of Harry Markowitz's mean-variance (MV) optimization model and the advent of modern portfolio theory marked a new era in the field of financial and investment decision-making [4].

As investors inherently seek to attain higher profits with lower risks, while higher profits frequently entail a larger degree of risk, the MV model aims to strike a delicate balance between return and risk, optimizing portfolios either by maximizing returns while keeping variance constant or by minimizing variance while maintaining expected returns. Nevertheless, persistent uncertainty in the predictability of financial markets [5] and excessive dependence on historical data, combined with the utilization of estimated parameters derived from historical data for real-world decision-making, have presented substantial challenges to the practical implementation of the MV model [6].

In recent years, the development of machine learning (ML) and deep learning (DL) models as powerful artificial intelligence (AI) tools, coupled with their growing adoption in financial affairs, has begun to address some of the aforementioned challenges. AI models primarily aid investors in enhancing decision-making by forecasting future market trends. Notable ML models, such as Random Forest (RF) [7–10] and eXtreme Gradient Boosting (XGBoost) [11,12], are now actively employed for stock market predictions. In the realm of DL, Long Short-Term Memory (LSTM) and Convolutional Neural Network (CNN) models have gained significant attention and are widely utilized for stock market forecasting [1,13–16].

Financial time series forecasting is widely recognized as a significant challenge due to the inherent characteristics of financial data, which include random fluctuations, non-linearity, instability, and noise [17–19]. Traditional statistical time series forecasting models, such as the Autoregressive Integrated Moving Average (ARIMA) [20], heavily rely on the stability assumption, which may not be applicable to financial market time series data, particularly regarding stock prices and returns [21–23]. Hence, ML and DL models, which can address these

* Corresponding author.
E-mail addresses: mahdi.ashrafzade@aut.ac.ir (M. Ashrafzadeh), mohammad.sadrani@tu-dresden.de (M. Sadrani), sarfaraz.hashemkhani@ucn.cl (S.H. Zolfani).

https://doi.org/10.1016/j.rineng.2025.106263
Received 12 February 2025; Received in revised form 10 July 2025; Accepted 11 July 2025
Available online 12 July 2025

prediction complexities more efficiently, have shown superior performance compared to statistical models [21,24].

For example, consider a scenario where a stock trading system is designed to determine the optimal asset allocation for daily trading using the conventional MV model. In such cases, the MV model typically relies on historical average returns as the expected return for each asset. However, it is evident that historical data alone cannot provide an accurate estimate of future returns in the short term [25], leading to potential errors in daily trading decisions. This underscores the significance of incorporating ML and DL models into asset return prediction and their integration with the foundational MV model. This importance and its positive impact on enhancing trading system performance have been shown in previous studies (e.g., [9,25–28]). For instance, Yu et al. [28] utilized the ARIMA model to predict stock returns for upcoming periods and incorporated these predictions to compute expected returns and covariance matrices for various portfolio optimization models. The results showcased the superiority of prediction-based portfolio optimization models over conventional approaches. Furthermore, Ma et al. [9] applied predicted returns from diverse ML and DL models to construct portfolios in two stages: initial stock preselection based on high predicted returns, followed by portfolio formation using selected stocks and their associated predicted returns, in conjunction with the MV and Omega ratio. Their results demonstrated that the models with superior stock return prediction capabilities improved performance when integrated with optimization models and applied to daily trading strategies.

In the present study, we take a step further by proposing a methodology to enhance stock return forecasting using ML and DL models, followed by the application of predicted returns in portfolio optimization. Our approach involves the utilization of various clustering models for stock grouping. Subsequently, ML and DL models are trained using the representative data of each cluster, enabling the prediction of returns for stocks within the respective cluster. This prediction methodology not only eliminates the need to train individual models for each stock, as in conventional methods, but also enhances model generalization, effectively addressing the issue of overfitting.

Overall, the contributions of this study are summarized as follows:

1) Introduction of a novel framework that integrates stock clustering with various ML and DL models to enhance stock return prediction, including the combination of three clustering methods [dynamic time warping (DTW), K-Means, and density-based spatial clustering of applications with noise (DBSCAN)] with four prediction models [long short-term memory (LSTM), convolutional neural network (CNN), random forest (RF), and eXtreme Gradient Boosting (XGBoost)]. After clustering stocks, each ML and DL model is trained using representative data derived by averaging features and target variables for assets within each cluster. Subsequently, the trained ML and DL models are applied to predict returns for stocks within the cluster.
2) Application of the proposed clustering-based return prediction methodology in daily portfolio optimization, in conjunction with the Mean-Variance with Forecasting (MVF) model.
3) Utilization of multivariate time series data together with various technical indicator features as input for the proposed forecasting models integrated with the clustering approach.
4) Demonstration of the superiority of the proposed methodology using data from the New York Stock Exchange's large-cap stocks, along with a comprehensive comparison with conventional methods in the existing literature.

The remainder of the paper follows this structure: Section 2 reviews prior studies on integrating machine learning and deep learning models in portfolio optimization. Section 3 details our proposed portfolio optimization model, covering data gathering and preprocessing, clustering and prediction methods, and the portfolio optimization framework. Section 4 presents experimental results on the test data, comparing them with existing literature methods. Section 5 concludes the paper and suggests potential directions for future studies.

2. Literature review

After introducing Harry Markowitz's mean-variance (MV) optimization model, several studies have focused on enhancing its practical applications (see [29] for a review). Recently, the emergence of machine learning (ML) and deep learning (DL) models has led to a stream of research aiming to leverage these advanced techniques in combination with the MV model, utilizing its fundamental principles while maximizing the strengths of ML and DL methods to improve overall performance (see Nazareth and Reddy, [30] for an in-depth review of the application of ML models in finance, especially their capacity to handle non-stationary financial data).

Moreover, the growing body of literature highlights the significant advantages of integrating machine learning and deep learning models into financial time series forecasting frameworks. These advanced models, such as Multilayer Perceptron (MLP), recurrent neural network (RNN), LSTM, Gated Recurrent Unit (GRU), CNN, and Temporal Convolutional Neural Network (TCN), have demonstrated superior performance in capturing complex, non-linear relationships and long-term dependencies in financial and commodity markets. For instance, Lee and Xia [31] systematically compared various deep learning algorithms for forecasting crude oil spot prices using futures prices with different maturities. Their findings revealed that deep learning models, particularly LSTM and GRU architectures, consistently outperformed traditional approaches across multiple prediction horizons, especially in handling the intricate dynamics and volatility of financial time series. Such evidence underscores the growing consensus that leveraging deep learning techniques can substantially enhance predictive accuracy and robustness in financial forecasting tasks.

Freitas et al. [25] introduced a portfolio optimization model that utilizes neural network prediction models to identify short-term investment opportunities. This model incorporates risk measures based on prediction errors and selects predictions with low and complementary pairwise error profiles, facilitating efficient diversification. In an evaluation using the Brazilian stock market, their model demonstrated superiority over both the MV model and market indexes, capitalizing on short-term opportunities and generating normal prediction errors despite the non-normality of stock return time series.

Deng and Min [32] pioneered the application of linear regression for stock selection from US and global markets within the MV framework. They introduced constraints to address practical concerns such as risk tolerance, turnover, and tracking error in portfolio formation. Their findings indicated that systematic tracking of error and risk tolerance led to increased portfolio returns. Chen et al. [33] proposed a diversified approach to portfolio selection, focusing on stock forecasting to enhance the performance of portfolio optimization models. Their model consists of two steps: the utilization of ML and DL models, including SVM, RF, LSTM, extreme learning machine (ELM), and BPNN, to select diversified stocks with high predicted returns, followed by the combination of predicted returns with the Modified mean-variance (MMV) model for optimal allocation. Empirical results, based on CSI 100 component stocks, demonstrated the superiority of their RF+MMV model over similar models and market indexes. Behera et al. [34] employed ML models such as RF, AdaBoost, Support Vector Regression (SVR), k-nearest neighbors (KNN), and artificial neural networks (ANN) for stock selection with higher predicted returns. They applied the mean–VaR model for portfolio optimization. Experimental results revealed that the mean–VaR model, coupled with AdaBoost predictions, outperformed other benchmark models.

Yun et al. [35] introduced a novel portfolio management framework based on deep learning, outperforming single-stage deep learning processes in all cases using ETF price datasets. Kim and Lee [36] addressed

forecast uncertainty as a risk factor in portfolio optimization. They proposed a modified Auxiliary Classifier Generative Adversarial Network (PredACGAN) with risk measurement capabilities, showcasing superior performance over conventional deep learning-based portfolio methods. Hoseinzade and Haratizadeh [37] leveraged CNN to extract features and predict stock market movements, enhancing prediction performance in directional movements of stock markets. Besides, Abolmakarem et al. [38] developed a predictive Multi-Period Multi--Objective Portfolio Optimization model (MPMOPO) that integrates LSTM for stock price prediction and demonstrated its effectiveness in handling real cases.

The literature also reflects efforts to enhance the forecasting capabilities of ML and DL models, followed by their application in portfolio optimization. For example, Chen et al. [12] introduced an improved firefly algorithm (IFA) for optimizing XGBoost hyperparameters, leading to improved stock price predictions. They employed these predictions for preselection before portfolio construction. Ma et al. [39] adopted an autoencoder (AE) neural network for feature extraction and LSTM for return prediction, using the past 60 days' logarithmic daily returns as input features. Their hybrid prediction model, integrating AE and LSTM, outperformed other models based on various performance metrics and was applied to portfolio optimization alongside a worst-case omega model. Moreover, reinforcement learning (RL) has been used in portfolio optimization, with demonstrated efficiency highlighted in studies by Almahdi and Yang [40,41] and Park et al. [42].

Notably, only a few studies in the literature have attempted to explore the synergies between clustering and prediction models to enhance prediction accuracy. For instance, Sáenz et al. [43] applied K-means clustering and alternative distance metrics to stock clustering, improving stock price and return predictions. LSTM and ARIMA were used as prediction models. Results demonstrated enhanced prediction accuracy by using additional data from relevant stocks, with LSTM outperforming ARIMA. While traditional statistical models such as ARIMA and its hybrids (e.g., ARIMA-Prophet) have demonstrated competitive accuracy in some domains [44], recent advances in machine learning and deep learning methods—especially those leveraging multivariate features and non-linear relationships—have generally shown superior predictive performance, particularly in forecasting complex and volatile financial time series.

Moreover, Li et al. [45] proposed a clustering-enhanced deep--learning framework for stock price prediction. They utilized LSTM, gated recurrent unit (GRU), and recurrent neural network (RNN) as deep learning models and introduced a novel Weighted Dynamic Time Warping (WDTW) called Logistic WDTW (LWDTW) for detecting stocks with similar price patterns. Incorporating LWDTW clustering with deep learning models improved stock price prediction accuracy.

The main distinctions between our paper and the two previous works are summarized as follows. Firstly, in this research, unlike previous

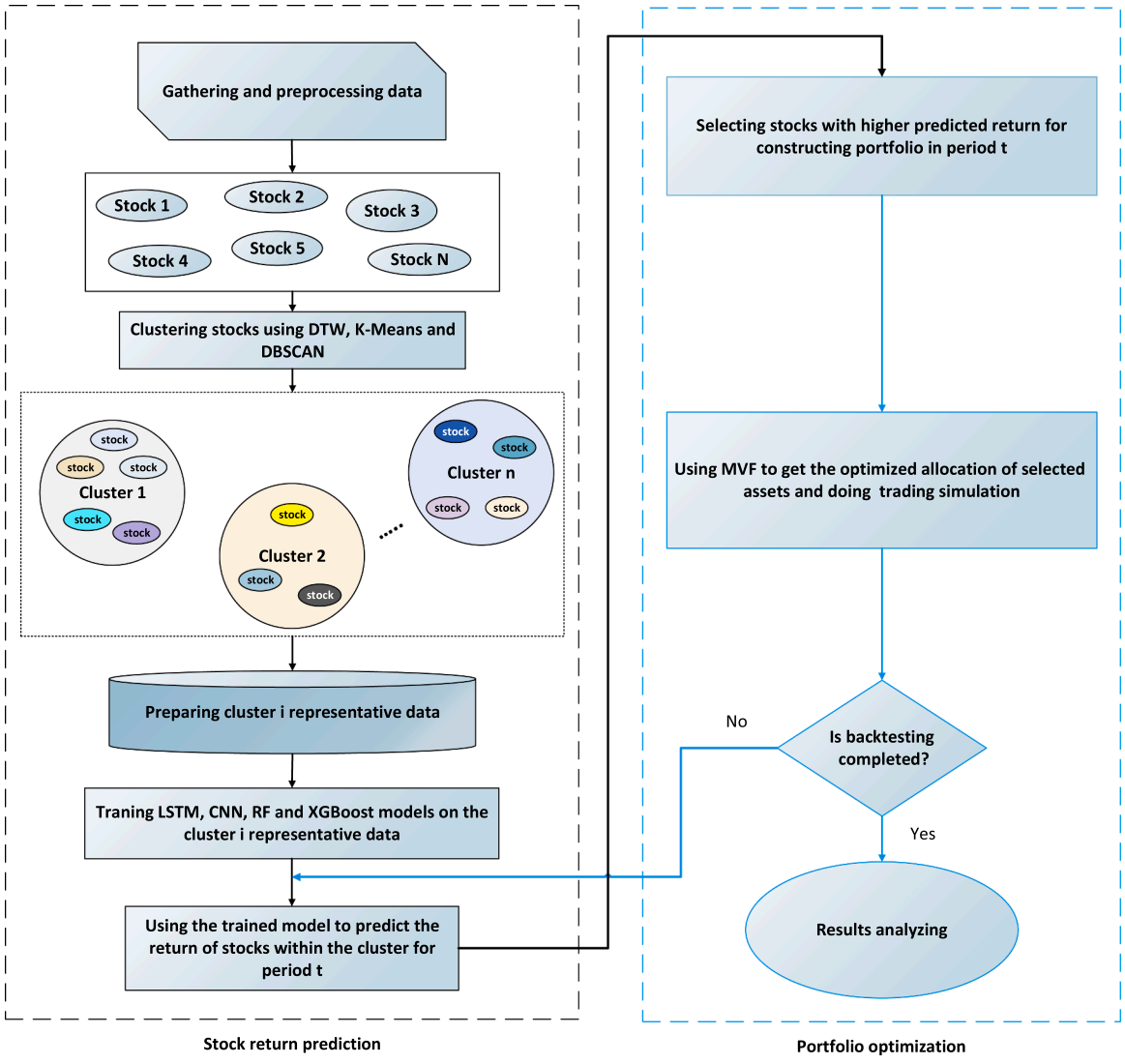

Fig. 1. Study flow chart.

studies, various machine learning and deep learning models are combined with different clustering models to enhance the prediction model's performance. We explore the combination of three clustering methods (DTW, K-Means, and DBSCAN) with four prediction models (LSTM, CNN, RF, and XGBoost). Secondly, after clustering the assets for each cluster, representative data for that cluster is calculated. Machine learning and deep learning models are then trained on this representative data, eliminating the need for a separate prediction model for each asset. Finally, in this research, improved forecasting models combined with asset clustering are integrated with portfolio optimization models for daily trading.

3. Methodology

In this part, we comprehensively describe the proposed portfolio optimization model, including data gathering and preprocessing, stock return prediction methodology, and portfolio formation methodology. Fig. 1 visually outlines the various stages of the model.

3.1. Data and input features

In this paper, we present experimental results based on a random selection of 17 large-cap stocks from the New York Stock Exchange (NYSE), with data collected over the period from 2013 to 2022. Table 1 presents various statistical attributes calculated from the daily returns of the selected stocks. Besides, Fig. 2 depicts the daily price variations of these selected stocks from 2013 to 2022.

One of the advantages of employing DL and ML models for time series prediction, in contrast to traditional models, is their ability to utilize multiple variables for forecasting the desired time series. In the context of stock prediction, technical indicators derived from price and volume data of stock transactions offer valuable insights into stock behavior.

In the initial stage of feature selection, a comprehensive set of technical indicators was considered, including Moving Average Convergence Divergence (MACD), Percentage Price Oscillator (PPO), Average True Range (ATR), Relative Strength Index (RSI), Stochastic Oscillator, Simple Moving Average (SMA), Exponential Moving Average (EMA), Bollinger Bands, and On-Balance Volume (OBV). Through a combination of trial-and-error experiments across different time periods and clusters, as well as correlation analysis to avoid multicollinearity, the most informative and non-redundant features were identified. Ultimately, six technical indicators—MACD, PPO, ATR, RSI, Stochastic Oscillator, and SMA—were selected for the final model, in line with previous studies such as Zhou et al. [46]. In addition, four lagged observations of the return (the target variable) were included as input features, following the approach of Paiva et al. [19]. Appendix A provides further details on the selected features. Before using the expressed features as input for DL and ML models, they are scaled by the following equation:

$$x^i_{scaled} = \frac{(x^i - x^i_{\min})}{(x^i_{\max} - x^i_{\min})} i = 1, \ldots, m \quad (1)$$

Where $x^i_{scaled}$is the scaled value of feature i, $x^i_{\min}$ and $x^i_{\max}$are the minimum and maximum values of feature i.

3.1.1. Training, testing, and validating phases

Given the time series nature of the data and its continuity properties, we adopt a rolling window approach for training our DL and ML models. Following the literature (e.g., [1,24]), each training and testing set is referred to as a study period. Each study period comprises a 750-day training phase followed by a 250-day testing phase. To ensure the robustness of our approach, we also introduce a validation dataset. Specifically, we train our models on the initial training set, and designate the following year as a validation set to optimize hyperparameters and mitigate overfitting issues. We then assess the model's performance on the data from the year after the validation period. For a visualization of this process, refer to Fig. 3, where the gray area encompasses all data samples from 2013 to 2022, each blue area represents a three-year training set, the green area represents the validation set for the following year, and the red area represents the testing set of the study period, which is the year immediately after the validation. This approach thus results in six study periods, each covering six years for testing.

3.2. Clustering methods

In this study, we selected DTW, K-Means, and DBSCAN as our primary clustering methods because each offers distinct advantages for financial time series. DTW effectively captures temporal similarities and aligns time series with possible phase shifts. K-Means is computationally efficient and groups stocks based on feature similarity, while DBSCAN is robust to noise and can detect clusters of arbitrary shapes, which is useful for complex financial data. These three methods together represent distance-based (DTW), centroid-based (K-Means), and density-based (DBSCAN) clustering paradigms, enabling a comprehensive assessment of clustering structures.

Here, we introduce the clustering models (DTW, K-Means, and DBSCAN) used at the beginning of each study period. We also describe the features and hyperparameters employed for each of these models.

Table 1
Summary statistics for selected assets.

	mean	std	min	25 %	50 %	75 %	max
PFE	0.0006	0.0134	-0.0773	-0.0060	0.0002	0.0069	0.1086
NVDA	0.0017	0.0273	-0.1876	-0.0110	0.0017	0.0146	0.2981
TSLA	0.0021	0.0358	-0.2106	-0.0149	0.0013	0.0190	0.2440
MA	0.0010	0.0170	-0.1273	-0.0071	0.0016	0.0091	0.1661
KO	0.0004	0.0112	-0.0967	-0.0045	0.0005	0.0059	0.0648
V	0.0009	0.0156	-0.1355	-0.0068	0.0013	0.0087	0.1384
JPM	0.0007	0.0170	-0.1496	-0.0074	0.0005	0.0089	0.1801
XOM	0.0004	0.0163	-0.1222	-0.0070	0.0001	0.0077	0.1269
SHW	0.0009	0.0160	-0.1868	-0.0068	0.0009	0.0085	0.1445
BA	0.0007	0.0236	-0.2385	-0.0088	0.0007	0.0102	0.2432
WMT	0.0005	0.0127	-0.1138	-0.0053	0.0005	0.0064	0.1171
UNH	0.0010	0.0157	-0.1728	-0.0068	0.0010	0.0084	0.1280
BRK-B	0.0006	0.0119	-0.0959	-0.0054	0.0005	0.0065	0.1161
AMZN	0.0010	0.0205	-0.1405	-0.0087	0.0009	0.0112	0.1575
MSFT	0.0010	0.0167	-0.1474	-0.0067	0.0007	0.0092	0.1422
GOOGL	0.0007	0.0168	-0.1163	-0.0069	0.0007	0.0090	0.1626
AMT	0.0007	0.0149	-0.1516	-0.0066	0.0009	0.0082	0.1222

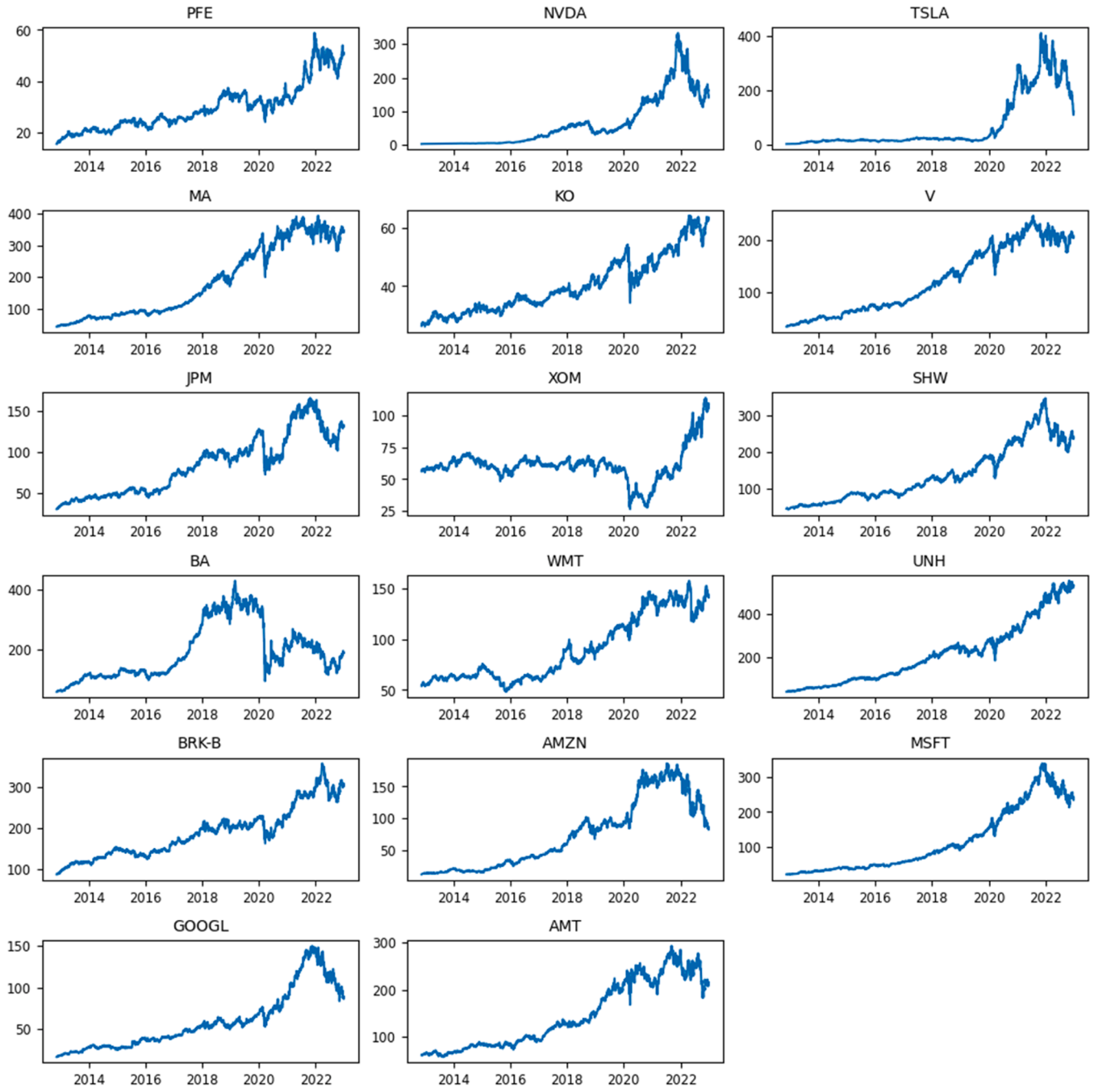

Fig. 2. Price variations of selected stocks from 2013 to 2022.

3.2.1. *Dynamic time warping (DTW)*

Dynamic Time Warping (DTW) is a well-established elastic distance measure extensively utilized for time series clustering. This method was first introduced by [47], and so far, it has been widely used for time series clustering in studies such as [43,48,49]. The base idea of this method is to calculate distance matrices between individual stock price time series and obtain optimal matches by minimizing distances between these time series [45]. In our study, DTW is employed as one of the clustering methods to group stocks based on their closing prices during the training set of each study period. To determine the optimal number of clusters, we calculate Silhouette's coefficient for various cluster numbers, typically ranging from 1 to 17. The aim is to balance the number of clusters and the Silhouette coefficient value [50]. This process is repeated at the beginning of each study period.

3.2.2. *K-Means clustering*

K-Means clustering, introduced by Forgy [51] and MacQueen [52], is a well-known machine-learning algorithm that groups data points into clusters based on their proximity to cluster centers. The algorithm starts with an initial clustering, and through iterations, it refines the clusters until convergence. The optimal number of clusters is determined at the beginning of each study period using the Silhouette coefficient. The features employed in the K-Means clustering process encompass the mean and variance of technical indicators (detailed in Appendix A) and stock returns computed over the training period of each study.

3.2.3. *Density-Based Spatial Clustering of Applications with Noise (DBSCAN)*

DBSCAN is an unsupervised machine-learning technique and a density-based clustering method introduced by Ester et al. [53].

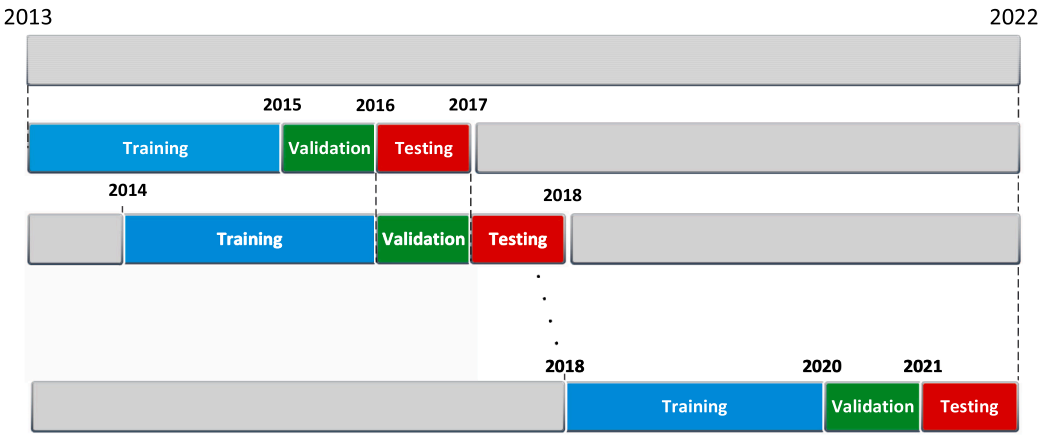

Fig. 3. Training, validation, and testing sets overlapping.

DBSCAN does not require specifying the number of clusters as a hyperparameter. Instead, it relies on two crucial hyperparameters: Minimum samples (MinPts) and ε (epsilon). In DBSCAN, a data point is considered a "core" point if it has at least MinPts neighbors within an ε radius. Core points close enough to each other (within the ε distance) are grouped into the same cluster.

For this study, a consistent MinPts value of 4 is applied throughout all study periods, as suggested by Ester et al. [53], who determined this value through the analysis of multiple datasets. To optimize the DBSCAN parameters for accurate clustering, the optimal ε is determined at the beginning of each study period. This is accomplished by fixing the number of neighbors at 4, calculating the average distance between each data point and its four nearest neighbors, and identifying the point where the plot of these distances exhibits a significant change. This point, often referred to as the "elbow point" or the first "valley" is chosen as the optimal ε. Besides, the features and criteria used for clustering stocks in this method are similar to those of the K-Means approach.

3.2.4. Clusters representative data

After the clustering process, we obtain representative data for each cluster. For example, we consider the calculation of the RSI time series as a feature for each stock within a cluster. The representative RSI time series for the cluster is computed by averaging the RSI values for all stocks within the cluster at each specific time point. This method is consistently applied to all features and the target variable, resulting in the generation of representative data for the entire cluster. The underlying assumption is that stocks within a cluster share a sufficient similarity, enabling the centroid or average values of features and targets to serve as representative data for the cluster. Besides, this approach enables us to reduce the amount of input to ML and DL models rather than considering all the data of assets inside a cluster, thus reducing the computational complexity of these models.

3.3. Prediction models

3.3.1. Long short-term memory (LSTM)

The LSTM (Long Short-Term Memory) neural network is a well-established and widely employed model in the context of time series forecasting. It falls under the Recurrent Neural Networks (RNNs) category and was initially introduced by Hochreiter and Schmidhuber [54]. LSTM addresses a common issue encountered in RNNs, known as the vanishing gradient problem, which arises when dealing with long sequences. To tackle this issue, LSTM is equipped with a memory cell to capture and retain temporal information from prior predictions, selectively propagating it through the network as needed. Key hyperparameters for LSTM include the number of hidden layers, neurons within each hidden layer, activation functions, optimizer, learning rate, epochs, batch size, and dropout rate (see Table 2 for a detailed overview of the specific values examined for each hyperparameter).

As previously mentioned, validation data is employed in each study period to fine-tune hyperparameters. After iterative testing, hyperparameters demonstrating consistently superior performance across multiple study periods are chosen. The network structure comprises two hidden layers with 32 and 64 neurons, a linear activation function, an Adam optimizer with a learning rate of 0.001, a dropout rate of 0.25, 160 epochs, and a batch size of 128.

3.3.2. Convolutional neural network (CNN)

The Convolutional Neural Network (CNN), initially introduced by Lecun et al. [55], is primarily utilized in image processing and machine vision. However, recent studies (such as [16,56,57]) have demonstrated its applicability in predicting stock price time series through one-dimensional (1D) CNNs. The core concept of CNN involves applying multiple filters to extract data features, employing pooling operations on input data, and generating output through one or more fully connected layers. Essential hyperparameters for 1D CNN in time series prediction include the number of convolutional layers, max-pooling layers, the number of filters in each convolutional layer, max-pooling layer pooling rate, fully connected layers, and their neurons, optimizer, learning rate, loss function, activation functions, epochs, and batch size. Table 3 shows the investigated parameter ranges for each of these hyperparameters.

The optimal architecture is derived through iterative testing on the validation data of study periods. It consists of two 1D convolutional layers, each with 128 and 32 filters. Following each convolutional layer is a 1D max-pooling layer with pooling sizes of 3 and 1, respectively. The selected hyperparameters include Adam as the optimizer, a learning rate of 0.005, a linear activation function, 60 epochs, and a batch size of 32. The output is processed through a fully connected layer with a single neuron and linear activation.

3.3.3. Random forest regressor (RF)

Random Forest (RF), introduced by Ho [58], is an ensemble and

Table 2
LSTM hyperparameters.

Hyperparameter	Value range
Hidden layer	1, 2, 3, …, 5
Number neurons	16, 32, 64, 128
Activation	linear, tanh, relu
Optimizer	Adam, Adamax, SGD
Loss function	Mean squared error
Learning rate	0.01, 0.001, 0.0005
Epochs	120, 160, 220
Batch size	32, 64, 128
Dropout rate	0.2, 0.25, 0.5

Table 3
CNN hyperparameters.

Hyperparameter	Value range
1D convolutional layers	1, 2, 3, …, 5
1D max-pooling layers	1, 2, 3, …, 5
number of filters	16, 32, 64, 128
pooling size	1,2,3
activation	linear, tanh, relu
optimizer	Adam, Adamax, SGD
loss function	Mean squared error
learning rate	0.01, 0.005, 0.0001
epochs	60, 120, 180
batch size	32, 64, 128
dropout rate	0.2, 0.25, 0.5

nonparametric machine learning model capable of handling regression and classification tasks. It has found application in stock prediction in various studies (e.g., [59–61]). The core idea behind the RF is to aggregate the outputs of multiple decision trees instead of relying on a single tree. This method includes key hyperparameters, such as the number of estimators (trees), minimum samples required to split an internal node (min-sample-split), minimum samples needed for a leaf node (min-samples-leaf), maximum tree depth (max-depth), and the number of features considered for the best split (max-features) (see Table 4 for the values considered for each hyperparameter).

The best hyperparameters for the RF are determined through extensive experimentation across multiple study periods. These include 400 estimators, a min-sample-split of 15, a min-samples-leaf of 10, a max-depth of 64, and a max-features value of 7.

3.3.4. XGBoost

The XGBoost, initially introduced by Chen [62], is an implementation of gradient-boosting decision trees (GBDT). This algorithm is known for its low computational complexity and fast execution, making it a proper choice for stock prediction studies [12,63].

Table 5 shows the hyperparameters of XGBoost along with their respective range values. Through extensive experimentation, the best values for the number of estimators, learning rate, max-depth, and gamma are found to be 300, 0.01, 10, and 0.005, respectively.

3.4. Portfolio optimization

3.4.1. Markowitz mean-variance model

The Mean-Variance (MV) model, introduced by Markowitz [4], serves as a pivotal solution for optimal portfolio selection and forms the foundation of Modern Portfolio Theory (MPT). Within this model, the quantification of investment return and risk is achieved through expected return and variance, respectively. Identifying the trade-off point is crucial for investors, as their utility typically aims to attain a high return with the lowest risk. However, in reality, a higher return frequently entails a larger degree of risk. A set of optimal points is established to address this challenge, each corresponding to a specific risk level, collectively forming the efficient frontier. The Markowitz model is commonly expressed as a mathematical programming model, outlined as follows:

Table 4
RF hyperparameters.

Hyperparameter	Value range
number of estimators	100, 200, …, 800
min-sample-split	5, 10, 15, ..., 25
min-samples-leaf	1, 5, 10, …, 20
max-depth	12, 16, 32, 64
max-features	3, 5, 7, 10

Table 5
XGBoost hyperparameters.

Hyperparameter	Value range
Number of estimators	200, …, 600
Learning rate	0.001, 0.005, 0.01, 0.05
Max-depth	5, 10, …, 20
Gamma	0.001, 0.005, 0.01

$$\begin{aligned} &Min \sum_{i,j=1}^{N} x_i.x_j\sigma_{i,j} \\ &Max \sum_{i=1}^{N} x_i.\mu_i \\ &\text{s.t} \begin{cases} \sum_{i=1}^{N} x_i = 1 \\ 0 \le x_i \le 1 \\ i = 1, ..., N \end{cases} \end{aligned} \tag{2}$$

where $x_i$is the proportion of asset i in portfolio, n is the number of assets in the portfolio, $\sigma_{i,j}$ is the covariance of asset i and j, as well as $\mu_i$denotes the predicted return of asset i.

3.4.2. Mean–variance with forecasted return (MVF)

Mean–variance with forecasting (MVF) is a model designed for the optimal allocation of selected assets in preparation for the next day's trading and portfolio rebalancing [9,28]. The model is outlined as follows:

$$\begin{aligned} &\min \sum_{i,j=1}^{N} x_i.x_j.\sigma_{i,j} - \sum_{i=1}^{N} x_i.\hat{r}_i - \sum_{i=1}^{N} x_i.\bar{e}_i \\ &\text{s.t.} \begin{cases} \sum_{i=1}^{N} x_i = 1 \\ 0 \le x_i \le 1 \\ i = 1, ..., N \end{cases} \end{aligned} \tag{3}$$

where N is the number of assets in the portfolio, $x_i$is the proportion of asset i in portfolio, $\hat{r}_i$ is the predicted return of asset i at time t, and $\sigma_{i,j}$is the covariance between asset i and j. $\bar{e}_i$ is the average prediction error of the asset i which is calculated for the last 20 days before the day t. Note that e_i is obtained by $e_i = r_i - \hat{r}$, where r_i is actual return of asset i. We employ a 20-day sample period for computing $\bar{e}_i$ and the covariance matrix, as used in Yu et al. [28] and Ma et al. [9].

In our framework, the predicted returns generated by the ML/DL models (such as LSTM, CNN, etc.) are directly used as the expected return inputs ($\hat{r}_i^t$) in the MVF optimization process for each asset at each time step. This integration enables the portfolio allocation to dynamically adjust based on the latest forecasts, thereby enhancing responsiveness to market changes. The portfolio is rebalanced on a daily basis following a rolling window approach, where the models are updated and the optimization is performed at the end of each trading day to determine asset weights for the next day. This fixed daily rebalancing schedule is consistent with prior studies [9,28] and allows for systematic evaluation of the predictive power of the ML/DL models within the portfolio optimization context.

4. Experimental results

In this section, we present the experimental results of our proposed model on the test data and compare them with existing methods in the literature. We first evaluate and compare the results of the proposed

clustering-based return prediction with conventional methods. Then, we analyze the outcomes of portfolio optimization based on the predicted returns.

4.2. Prediction results

To determine the effectiveness of different models in return prediction, their performance is evaluated from various perspectives. Mean Squared Error (MSE) and Mean Absolute Error (MAE) are expressed in Eqs (4) and (5), where r_i represents actual return and $\hat{r}_i$ represents predicted return at the time i. These metrics, which have been used in several studies (e.g., [12,24,39]), quantify the positive error incurred by a model in predicting each point. A lower value for these metrics indicates a higher level of success in forecasting for a model.

Moreover, H_R, H_{R+}, and H_{R-}, in Eqs (6-8), denote the total hit rate, the accuracy of positive predictions, and the accuracy of negative predictions, respectively. These metrics, known as accuracy metrics, assess a model's ability to predict the direction of the return by comparing predicted and actual values [9,25]. The higher value indicates the model's optimal performance in detecting the return's direction.

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(v)^2 \tag{4}$$

$$MAE = \frac{1}{n}\sum_{i=1}^{n}|r_i - \hat{r}_i| \tag{5}$$

$$H_R = \frac{Count_{i=1}^{n}(r_i\hat{r}_i > 0)}{Count_{i=1}^{n}(r_i\hat{r}_i \neq 0)} \tag{6}$$

$$H_{R+} = \frac{Count_{i=1}^{n}(r_i > 0 and \hat{r}_i > 0)}{Count_{i=1}^{n}(\hat{r}_i > 0)} \tag{7}$$

$$H_{R-} = \frac{Count_{i=1}^{n}(r_i < 0 and \hat{r}_i < 0)}{Count_{i=1}^{n}(\hat{r}_i < 0)} \tag{8}$$

Table 6 presents the prediction results for each clustering-based return prediction model and conventional method over the test data. A comparison of the clustering method with ML and DL models trained on clusters derived from the K-means clustering method reveals superior performance in prediction metrics. In contrast to training ML and DL models for each stock individually, each clustering-based model demonstrated better results. K-Means+LSTM exhibited the lowest MSE and MAE among all prediction models. Moreover, K-Means+LSTM showed the lowest standard deviation, indicating stability in its performance for predicting the return of different stocks.

To provide a comprehensive evaluation, we compared the total execution time required for both clustering-based and non-clustering models. The total execution time includes both the time required for clustering (in clustering-based approaches) and the time required for model training and preparation for prediction.

In our study, the dataset was divided into six study periods from 2013 to 2022. In each period, the models were updated from scratch—meaning that clustering was performed anew and all models were retrained specifically for that period. Therefore, the reported execution times represent the average total execution time for each model type across all six study periods.

As shown in Table 6, clustering-based models require substantially less total execution time compared to non-clustering models. In the non-clustering approach, a separate model must be trained for each of the 17 stocks, which significantly increases the computational burden. In contrast, clustering-based models only require training one model per cluster, in addition to the relatively minor overhead of the clustering step.

All experiments were conducted on a system equipped with an Intel® Core™ i7-6700HQ CPU @ 2.60GHz. While absolute execution times

Table 6
The predictive performance over three clustering-based and conventional methods.

		MSE	MAE	HR	HR+	HR-	Total execution time (min)
DTW + LSTM	**Mean**	0.00044	0.01401	50.74585	49.87762	51.68365	32
	SD	0.00040	0.00503	0.88641	5.22500	5.13900	
DTW + CNN	**Mean**	0.00048	0.01423	50.55605	55.00417	45.41901	27
	SD	0.00036	0.00475	1.69863	4.31798	2.90023	
DTW + RF	**Mean**	0.00049	0.01441	51.17514	55.49084	46.16264	28
	SD	0.00038	0.00501	1.01904	3.69900	3.37751	
DTW + XGBOOST	**Mean**	0.00049	0.01456	51.15455	60.33629	40.60676	24
	SD	0.00038	0.00501	1.31542	4.01401	4.14128	
DBSCAN + LSTM	**Mean**	0.00046	0.01413	50.79680	49.16519	52.25675	26
	SD	0.00038	0.00512	1.02062	5.16555	4.71075	
DBSCAN + CNN	**Mean**	0.00049	0.01423	50.2492	55.53045	43.83776	25
	SD	0.00036	0.00485	1.22317	4.08286	3.77977	
DBSCAN + RF	**Mean**	0.00051	0.01460	51.34253	56.78451	45.03232	27
	SD	0.00046	0.00563	1.76652	3.07872	4.28622	
DBSCAN + XGBOOST	**Mean**	0.00057	0.01528	51.09504	62.28626	38.50212	23
	SD	0.00053	0.00594	1.37254	3.47661	4.01213	
K-Means + LSTM	**Mean**	0.00042	0.01362	50.71118	46.11034	53.58668	35
	SD	0.00027	0.00406	1.00009	4.30981	3.65011	
K-Means + CNN	**Mean**	0.00047	0.01421	50.48248	54.93708	45.26437	27
	SD	0.00035	0.00470	1.16742	2.06929	1.78863	
K-Means + RF	**Mean**	0.00047	0.01419	51.43419	59.13851	42.70276	30
	SD	0.00035	0.00475	0.85826	2.73630	3.25381	
K-Means + XGBOOST	**Mean**	0.00048	0.01430	50.88883	60.83451	39.54514	25
	SD	0.00036	0.00484	1.14350	2.06730	2.67752	
LSTM	**Mean**	0.00070	0.01700	49.96575	49.85679	50.17638	120
	SD	0.00049	0.00562	1.92666	5.74146	5.58889	
CNN	**Mean**	0.00056	0.01507	50.22828	57.02635	42.38257	102
	SD	0.00035	0.00500	1.61885	5.53516	5.18243	
RF	**Mean**	0.00062	0.01601	50.68791	55.43461	45.15679	116
	SD	0.00055	0.00644	1.40939	4.37319	4.53554	
XGBOOST	**Mean**	0.00078	0.01765	50.04636	60.49021	39.43153	97
	SD	0.00069	0.00710	1.15411	4.85440	4.58590	

may vary depending on hardware and implementation, the relative efficiency gain from clustering remains clear and consistent. This substantial reduction in computational time highlights the practical advantage of clustering-based approaches, especially when scaling to larger portfolios or requiring frequent retraining.

Given the significance of MSE and MAE in portfolio formation using return prediction, as highlighted in Ma et al. [9], these metrics serve as crucial indicators. Fig. 4 compares the MSE and MAE of different models. The spread of errors is more noticeable with conventional models without clustering.

We utilize the Diebold-Mariano (DM) statistical test to validate the enhanced performance of our proposed method, which involves the comparison of two groups. The first group comprises the performance of each ML and DL prediction model individually trained on stocks. The second group encompasses models trained on stock clusters, applied to predict returns within those clusters. Examining the p-values in Table 7 for each asset, we observe that in most cases, the p-values fall below 5 %, indicating the disparity in performance between the two groups. While the clustering-based approach generally leads to significant improvements in prediction accuracy, as confirmed by the Diebold-Mariano test for most assets, there are occasional exceptions. For assets with highly unique or idiosyncratic return patterns, or during periods of market instability, clustering may not always provide statistically significant benefits. Nonetheless, the overall results indicate that clustering enhances predictive performance for the majority of assets and market scenarios by leveraging shared information among similar assets. This highlights the practical value of incorporating clustering in financial time series prediction, particularly for building more robust and generalizable models.

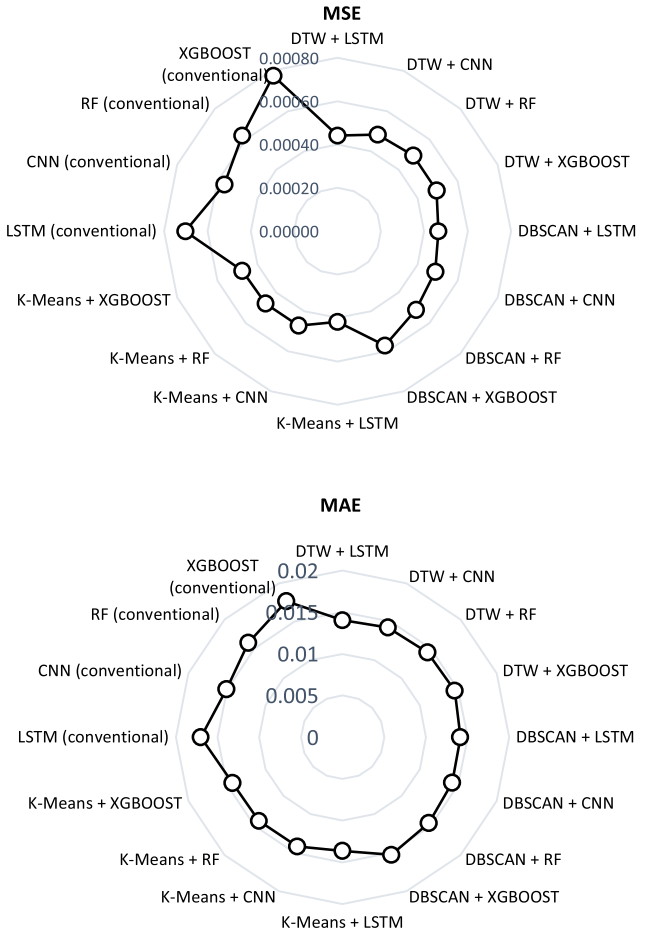

Fig. 4. MSE and MAE spider plots.

4.3. Details of financial performance

Overall, our results show that the prediction models based on the K-Means algorithm exhibit superior performance compared to the other two clustering models. The comparison includes various performance metrics, including expected return (ER), standard deviation (SD), value at risk (VaR), conditional value at risk (CVaR), coefficient of variation (CV), maximum drawdown (MD), sharp ratio, and Sortino ratio. Thus, we use and compare the combination of ML and DL models with K-Means against traditional ML and DL models in the context of daily trading portfolio construction. To provide further information, the subsequent sub-section includes the comparison of different portfolio models using these metrics, both daily and annually.

4.3.1. Model performance without transaction cost

Table 8 shows the result of applying models in portfolio optimization during the test period without considering any transaction cost. Panel A of this table presents daily return and risk metrics, while Panel B displays annualized return and risk metrics. According to the results, K-Means+LSTM+MVF exhibits the highest expected return among the models. The subsequent model with the highest expected return is LSTM+MVF, underscoring the significance of the LSTM prediction model. Analyzing risk factors such as SD, VaR, CVaR, and MD reveals that K-Means+LSTM+MVF demonstrates the best results, with its risk factors being the lowest among the models.

In our work, the risk-free return is set at 3.83 %, based on the US average 7-year treasury rate, and is used to calculate the annual sharp and Sortino ratio. Panel B of Table 8 shows the K-Means+LSTM+MVF has the highest sharp and Sortino ratio. By comparing each of the clustering-based models with its conventional form, for instance, the K-Means+CNN +MVF with CNN +MVF, K-Means+RF+MVF with RF+MVF and K-Means+XGBOOST +MVF with XGBOOST +MVF, it can be seen that the clustering-based type outperforms in terms of most return and risk criteria. An essential factor highlighted in Panel A of Table 8 is the average turnover directly associated with transaction costs. Therefore, the model with a lower average turnover will have a lower transaction cost. The K-Means+LSTM +MVF has the lowest average turnover.

4.3.2. Model performance with transaction cost

The transaction cost is calculated by equation (9) (Zhang et al., 2021):

$$R_{i,t} = w_{i,t-1} r_{i,t} - C\left|w_{i,t}\left(1 + w_{i,t-1} r_{i,t}\right) - w_{i,t-1}\right|$$
$$R_{p,t} = \sum_{i=1}^{N} R_{i,t} \tag{9}$$

where the $R_{i,t}$,$r_{i,t}$, $w_{i,t}$, and $R_{p,t}$ are trade return, price return, the ratio of stock I, and portfolio return at the period t, respectively. Moreover, C represents the cost rate, denoted in basis points (1bp = 10−4).

To simplify the analysis, we adopt the basis points (bps) framework, which aggregates various transaction-related costs—including brokerage fees, slippage costs, and the impact of daily portfolio weight changes—into a single metric [64,65]. In this study, the transaction cost reflects the reduction in portfolio returns caused by daily asset weight changes. Accordingly, we do not separate individual transaction cost components.

Analysis of the experimental results reveals that the transaction cost associated with large market stocks in the NYSE is a minimum of 20 basis points [64,65]. The cost rates are set at 10 and 20 bps in our study. The simulation results for the respective cost rates of 10 and 20 bps are presented in Tables 9 and 10.

Table 9 indicates that, except for the CNN+MVF and RF+MVF models, all other models demonstrate positive expected returns (ER). Notably, the K-Means+LSTM+MVF and the LSTM+MVF models exhibit the highest Ers. Turning to Table 10, which considers a cost rate of 20

Table 7
Diebold Mariano test result between clustering-based prediction models and conventional methods.

Statistical factor:	P-value											
Group1 conventional:	**LSTM**			**CNN**			**RF**			**XGBOOST**		
Group2 Clustering-based:	**DTW+ LSTM**	**DBSCAN + LSTM**	**K-Means + LSTM**	**DTW+ CNN**	**DBSCAN + CNN**	**K-Means + CNN**	**DTW+ RF**	**DBSCAN + RF**	**K-Means + RF**	**DTW+ XGBOOST**	**DBSCAN + XGBOOST**	**K-Means + XGBOOST**
PFE	0.01	0.00	0.03	0.01	0.00	0.01	0.22	0.00	0.00	0.00	0.00	0.00
NVDA	0.00	0.00	0.00	0.01	0.26	0.00	0.00	0.00	0.00	0.00	0.00	0.00
TSLA	0.18	0.33	0.00	0.88	0.93	0.10	0.00	0.00	0.00	0.00	0.00	0.00
MA	0.00	0.00	0.00	0.25	0.06	0.03	0.00	0.01	0.00	0.00	0.00	0.00
KO	0.00	0.00	0.00	0.02	0.21	0.41	0.40	0.50	0.92	0.00	0.33	0.00
V	0.00	0.00	0.00	0.26	0.08	0.95	0.00	0.00	0.00	0.00	0.00	0.00
JPM	0.00	0.00	0.00	0.15	0.32	0.87	0.50	0.27	0.76	0.00	0.00	0.00
XOM	0.00	0.00	0.00	0.00	0.12	0.99	0.93	0.00	0.00	0.00	0.00	0.00
SHW	0.00	0.00	0.00	0.39	0.46	0.71	0.00	0.00	0.00	0.00	0.00	0.00
BA	0.77	0.57	0.11	0.38	0.01	0.01	0.00	0.00	0.00	0.00	0.00	0.00
WMT	0.60	0.01	0.08	0.00	0.28	0.00	0.00	0.00	0.00	0.00	0.00	0.00
UNH	0.03	0.10	0.02	0.06	0.00	0.00	0.26	0.10	0.23	0.01	0.00	0.00
BRK-B	0.00	0.00	0.00	0.00	0.02	0.00	0.11	0.23	0.09	0.00	0.00	0.00
AMZN	0.02	0.00	0.00	0.75	0.67	0.75	0.00	0.00	0.00	0.00	0.00	0.00
MSFT	0.01	0.00	0.01	0.36	0.10	0.02	0.00	0.00	0.01	0.00	0.00	0.00
GOOGL	0.62	0.01	0.53	0.01	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
AMT	0.68	0.35	0.01	0.09	0.16	0.01	0.00	0.00	0.00	0.00	0.00	0.00

Table 8
Performance of different models without transaction cost.

Model	**K-Means+ LSTM +MVF**	**K-Means+ CNN +MVF**	**K-Means+ RF +MVF**	**K-Means+ XGBOOST +MVF**	**LSTM+MVF**	**CNN+MVF**	**RF+MVF**	**XGBOOST+MVF**
Panel A: Daily return & risk measures								
ER	0.0022	0.0010	0.0012	0.0010	0.0015	0.0006	0.0006	0.0010
Min	-0.1105	-0.1854	-0.2385	-0.1382	-0.2113	-0.2385	-0.1238	-0.1474
Max	0.1978	0.2038	0.2004	0.1429	0.1964	0.2432	0.1879	0.1964
Kurtosis	11.3759	24.6492	19.4366	7.2146	19.5160	32.4772	8.9493	7.7639
SD	0.0200	0.0201	0.0239	0.0218	0.0228	0.0227	0.0222	0.0257
VaR (1 %)	0.0443	0.0459	0.0545	0.0497	0.0515	0.0522	0.0511	0.0588
CVaR (1 %)	0.0651	0.0816	0.0874	0.0752	0.0840	0.0935	0.0713	0.0799
VaR (5 %)	0.0307	0.0322	0.0382	0.0349	0.0360	0.0367	0.0359	0.0413
CVaR (5 %)	0.0501	0.0604	0.0680	0.0587	0.0588	0.0671	0.0545	0.0609
CV	9.1243	20.7909	19.7589	21.7501	15.6825	39.0965	36.4036	26.4221
MD	1.5587	1.9097	2.1903	1.9674	2.0757	1.9807	1.6589	1.7504
Avg. Turnover	0.3110	0.8145	0.8228	0.7085	0.3291	0.7805	0.8093	0.7598
Panel B: Annualized return & risk measures								
ER	0.7272	0.2739	0.3536	0.2848	0.4375	0.1560	0.1648	0.2750
SD	0.3157	0.3185	0.3786	0.3449	0.3602	0.3586	0.3514	0.4061
Sharp	2.3034	0.8600	0.9340	0.8258	1.2146	0.4351	0.4691	0.6770
Sortino	2.4910	0.9383	1.2112	0.9757	1.4985	0.5344	0.5646	0.9418

bps, we see that all models, except for the K-Means+LSTM+MVF and LSTM+MVF models, display negative expected returns. Considering the risk factors presented in panel A, namely SD, VaR, CVaR, and MD, alongside the annualized return and risk factors in panel B of both Tables 9 and 10, the K-Means+LSTM+MVF and LSTM+MVF models consistently emerge as the superior models for portfolio selection. In summary, among the various ML and DL prediction models, the LSTM proves to be the more favorable choice. Furthermore, given that the K-Means+LSTM+MVF model outperforms the LSTM+MVF model across all scenarios, it can be concluded that the proposed clustering-based model holds the advantage.

4.3.3. *Visualization of model performances*

Three types of diagrams are employed to illustrate the performance of different models to facilitate further comparisons. The first type of diagram is cumulative return diagrams, presented in Figs. 5–7. Fig. 5 represents the cumulative return for each model without considering transaction costs. Notably, K-Means+LSTM+MVF exhibits the highest cumulative return, followed by LSTM+MVF and K-Means+RF+MVF.

Upon including transaction costs of 10 bps and 20 bps, as depicted in Figs. 6 and 7, respectively, the cumulative return of each model experiences a significant decrease. Analyzing the graphs reveals that, when considering transaction costs, the cumulative return of K-Means+LSTM+MVF and LSTM+MVF will decrease to a lesser extent than the other models. In fact, with a transaction cost of 20 bps, in Fig. 7, all models, except K-Means+LSTM+MVF and LSTM+MVF, result in a negative cumulative return. Among these models,

K-Means+LSTM+MVF maintains the highest positive cumulative return, followed by LSTM+MVF.

Another type of diagram employed is the return-risk ratio histogram to assess whether a model's performance is consistent across different periods or performs well only during specific intervals. Figs. 8–10 illustrate the return-risk ratio histograms for each model at six-month intervals throughout the test period.

Fig. 8 reveals that almost all models demonstrate positive return-risk ratios in all 6-month periods, except for the last two intervals.

Table 9
Performance of different models considering transaction cost (=10 bps).

Model	K-Means+ LSTM +MVF	K-Means+ CNN +MVF	K-Means+RF +MVF	K-Means+ XGBOOST +MVF	LSTM+MVF	CNN+MVF	RF+MVF	XGBOOST+MVF
Panel A: Daily return & risk measures								
ER	0.0019	0.0002	0.0004	0.0003	0.0011	-0.0002	-0.0002	0.0002
Min	-0.1112	-0.1864	-0.2388	-0.1382	-0.2123	-0.2385	-0.1243	-0.1474
Max	0.1975	0.2037	0.2001	0.1419	0.1964	0.2432	0.1869	0.1944
Kurtosis	11.3994	24.4765	19.4543	7.1965	19.6236	32.4605	8.9103	7.6384
SD	0.0200	0.0202	0.0239	0.0218	0.0228	0.0227	0.0222	0.0257
VaR (1 %)	0.0446	0.0468	0.0553	0.0505	0.0519	0.0529	0.0520	0.0595
CVaR (1 %)	0.0664	0.0823	0.0878	0.0758	0.0844	0.0938	0.0712	0.0795
VaR (5 %)	0.0310	0.0330	0.0390	0.0356	0.0364	0.0375	0.0368	0.0420
CVaR (5 %)	0.0509	0.0613	0.0685	0.0594	0.0591	0.0670	0.0555	0.0612
CV	10.6560	123.7491	63.4566	70.1873	20.2957	-115.9841	-109.7864	115.8361
MD	1.5629	1.9153	2.1935	1.9743	2.0809	1.9807	1.6649	1.7581
Panel B: Annualized return & risk measures								
ER	0.5972	0.0416	0.0989	0.0809	0.3241	-0.0477	-0.0494	0.0570
SD	0.3159	0.3189	0.3787	0.3452	0.3606	0.3585	0.3517	0.4059
Sharp	1.8906	0.1304	0.2613	0.2342	0.8989	-0.1331	-0.1404	0.1403
Sortino	2.0533	0.1430	0.3401	0.2780	1.1144	-0.1640	-0.1699	0.1959

Table 10
Performance of different models considering transaction cost (=20 bps).

Model	K-means+ LSTM +MVF	K-Means+CNN +MVF	K-Means+RF +MVF	K-Means+XGBOOST +MVF	LSTM+MVF	CNN+MVF	RF+MVF	XGBOOST+MVF
Panel A: Daily return & risk measures								
ER	0.0016	-0.0006	-0.0005	-0.0004	0.0008	-0.0010	-0.0010	-0.0005
Min	-0.1118	-0.1874	-0.2391	-0.1382	-0.2133	-0.2385	-0.1248	-0.1474
Max	0.1971	0.2035	0.1998	0.1409	0.1964	0.2432	0.1859	0.1924
Kurtosis	11.4134	24.2543	19.4386	7.1531	19.7153	32.3939	8.8384	7.4948
SD	0.0200	0.0202	0.0240	0.0219	0.0228	0.0227	0.0223	0.0257
VaR (1 %)	0.0449	0.0476	0.0562	0.0513	0.0523	0.0537	0.0529	0.0603
CVaR (1 %)	0.0678	0.0831	0.0872	0.0757	0.0848	0.0941	0.0719	0.0799
VaR (5 %)	0.0313	0.0339	0.0399	0.0364	0.0368	0.0383	0.0377	0.0428
CVaR (5 %)	0.0512	0.0628	0.0701	0.0601	0.0587	0.0676	0.0564	0.0621
CV	12.8069	-31.1500	-51.8043	-56.0260	28.7210	-23.2871	-21.7045	-47.9156
MD	1.5671	1.9209	2.1967	1.9812	2.0862	1.9807	1.6710	1.7660
Panel B: Annualized return & risk measures								
ER	0.4768	-0.1497	-0.1092	-0.0930	0.2197	-0.2162	-0.2265	-0.1254
SD	0.3160	0.3194	0.3789	0.3458	0.3610	0.3585	0.3524	0.4059
Sharp	1.5087	-0.4687	-0.2883	-0.2689	0.6088	-0.6029	-0.6428	-0.3089
Sortino	1.6436	-0.5161	-0.3765	-0.3206	0.7575	-0.7452	-0.7809	-0.4323

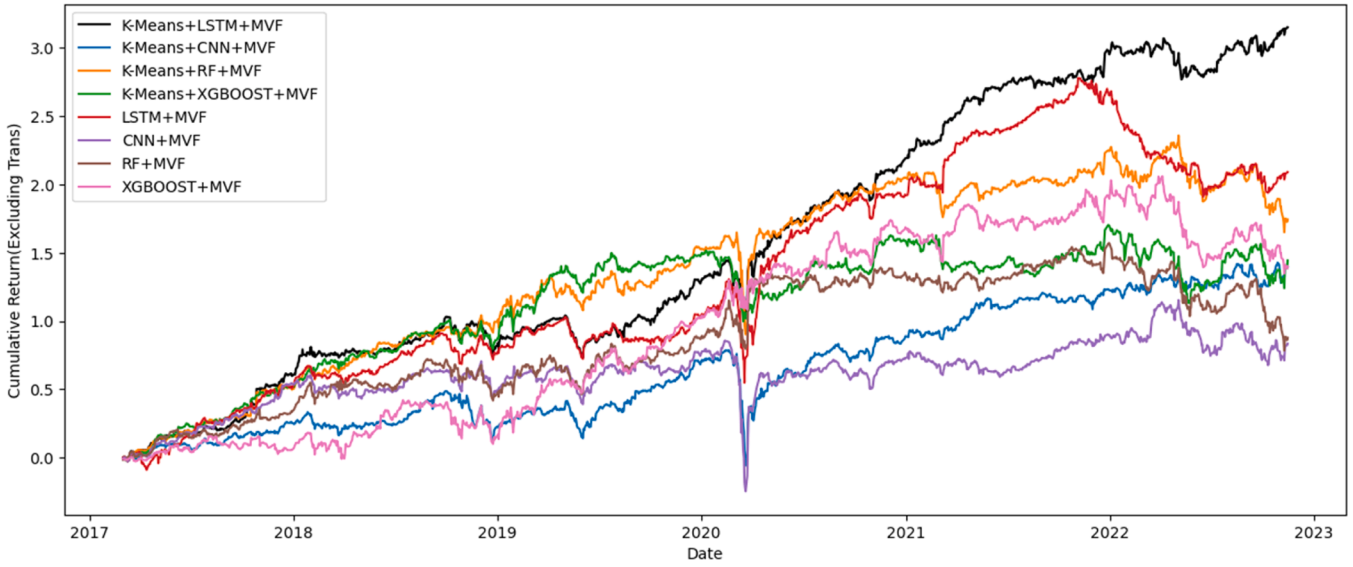

Fig. 5. Cumulative return without considering transaction cost.

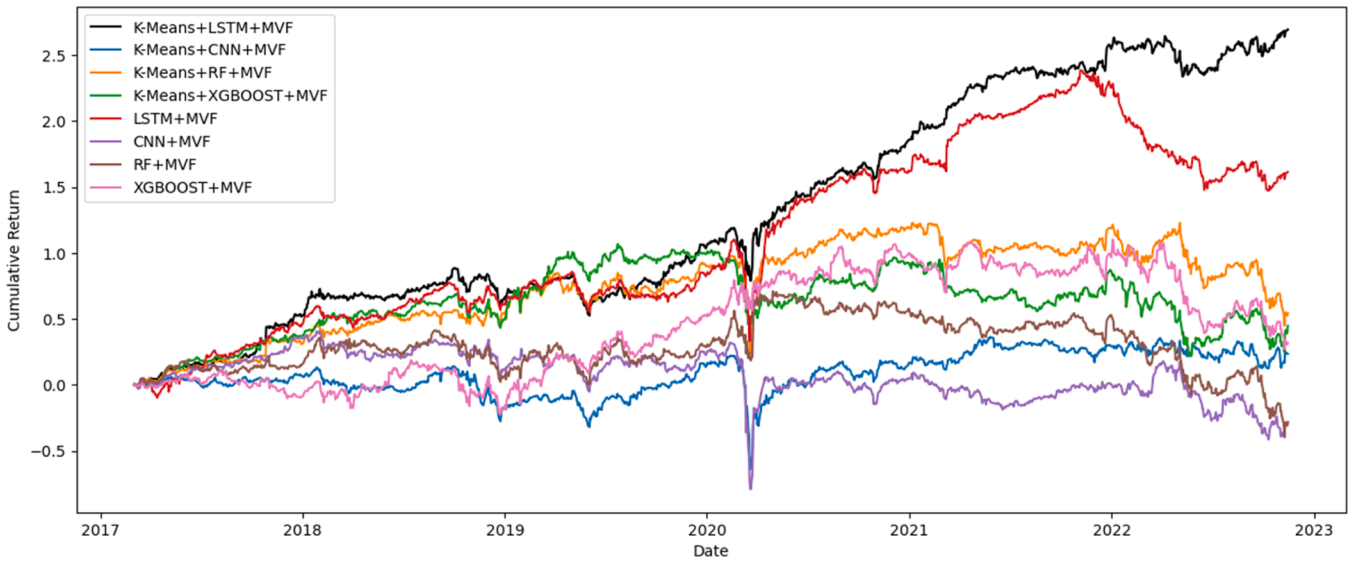

Fig. 6. Cumulative return including transaction cost (10bps).

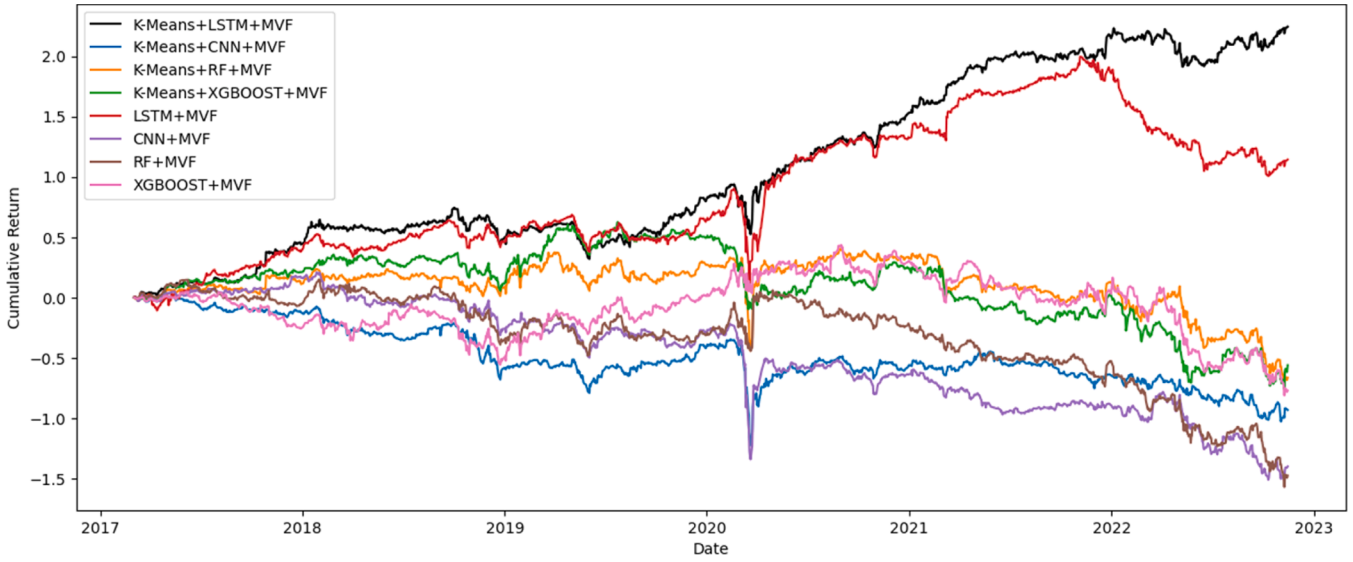

Fig. 7. Cumulative return, including transaction cost (20bps).

Conversely, as can be seen in Figs. 9 and 10, some models display negative risk-return ratios when accounting for transaction costs of 10 and 20 basis points (bps).

However, Fig. 10, incorporating a transaction cost of 20 bps, highlights the superior performance of K-Means+LSTM+MVF and LSTM+MVF. These models outperform others by yielding only three negative return-risk ratios in the last 13 six-month intervals surveyed. In summary, K-Means+LSTM+MVF exhibits a favorable return-risk ratio, followed closely by LSTM.

Finally, the violin plots (in Figs. 11–13) reveal that portfolio selection models with clustering exhibit lower volatility and higher expected returns than those without clustering. This graphical representation is derived from the 6-month returns of various portfolio selection models.

5. Conclusions

This paper introduces an enhanced framework for daily portfolio construction through return prediction, aiming to improve accuracy. We employ three clustering techniques to categorize stocks, including DTW, K-Means, and DBSCAN. After clustering, representative data for each cluster is derived by averaging features and target variables for assets within the cluster. Machine Learning (ML) and Deep Learning (DL) models, including RF, XG Boost, LSTM, and CNN, are then trained on these cluster representative data. Notably, the K-Means+LSTM model exhibits superior forecasting performance.

As a key innovation in this study, instead of training separate models for each asset, a single model trained on the cluster's representative data is used to predict the return for all stocks within that cluster. Results, validated by DM tests, demonstrate that models combined with clustering surpass those trained individually for each asset, with the K-Means+LSTM model showing the most effectiveness.

We apply the MVF model to build daily trading portfolios based on predicted returns, selecting assets with the highest forecasted returns. Simulation results, considering transaction costs of 10 and 20 bps, highlight the K-Means+LSTM+MVF model and LSTM+MVF model as the top performers regarding return and risk metrics for daily and annual periods. Notably, the K-Means+LSTM+MVF model demonstrates minimal performance loss considering transaction costs. This framework

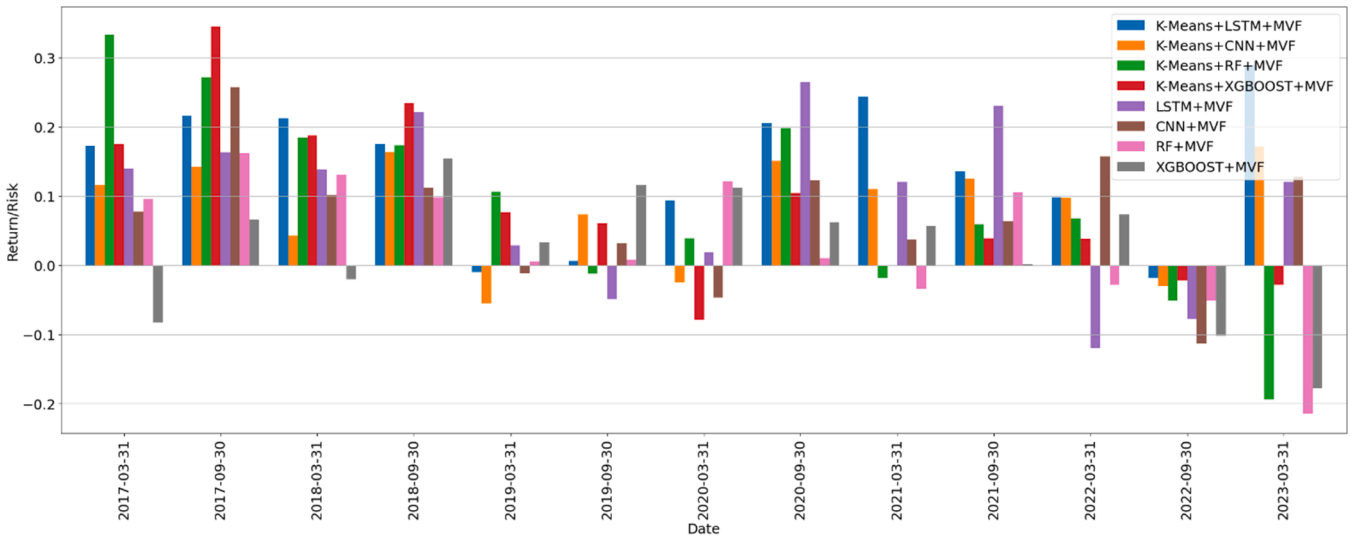

Fig. 8. Return–risk ratio of each six-month during 2017-2022 without considering transaction cost.

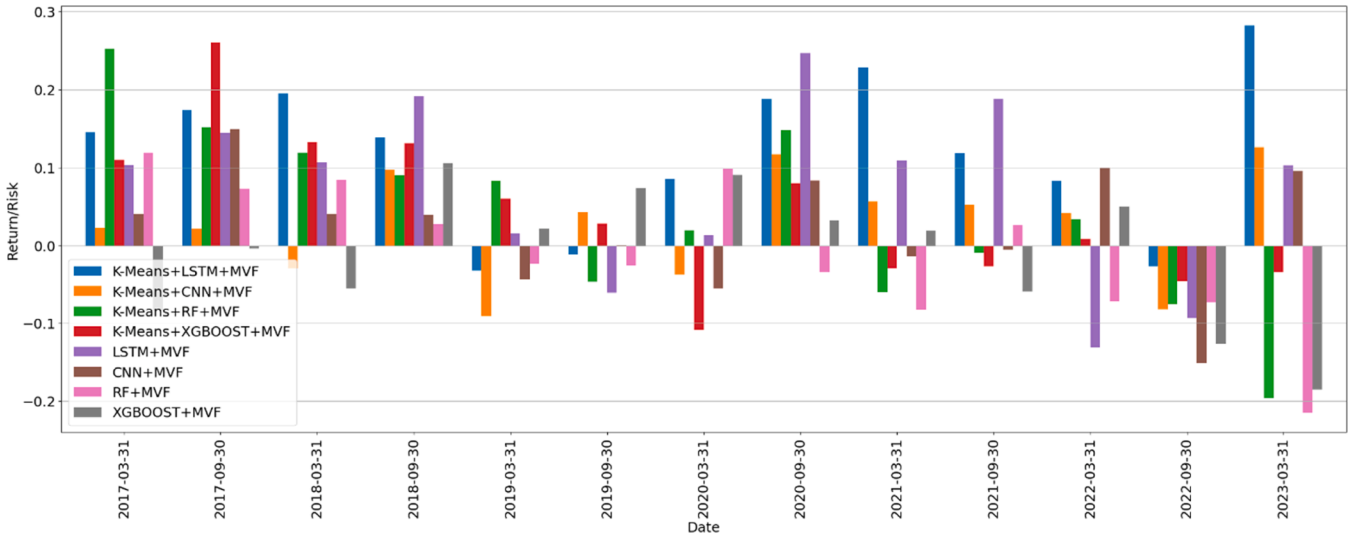

Fig. 9. Return–risk ratio of each six-month during 2017-2022, including transaction cost (10bps).

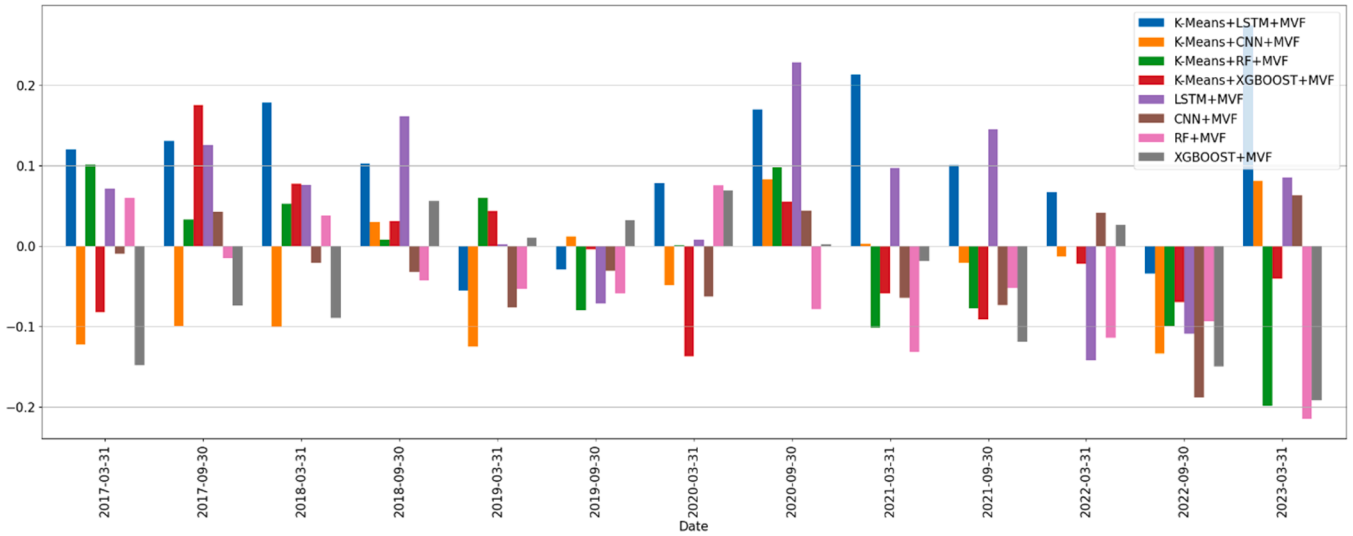

Fig. 10. Return–risk ratio of each six-month during 2017-2022, including transaction cost (20bps).

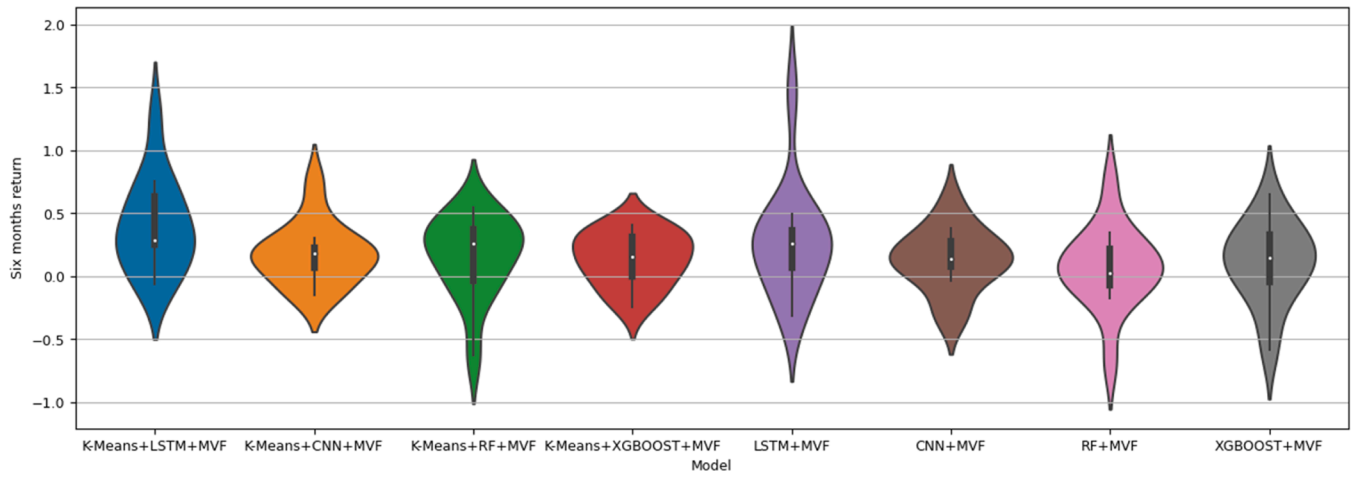

Fig. 11. Violin plot of daily returns without considering transaction cost.

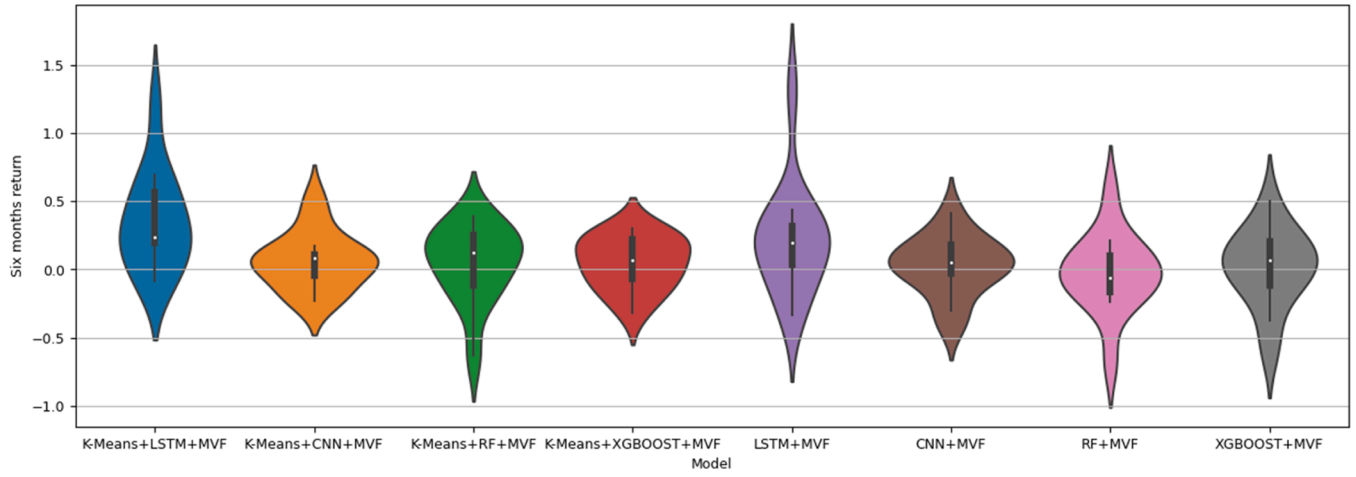

Fig. 12. Violin plot of daily returns, including transaction cost (10bps).

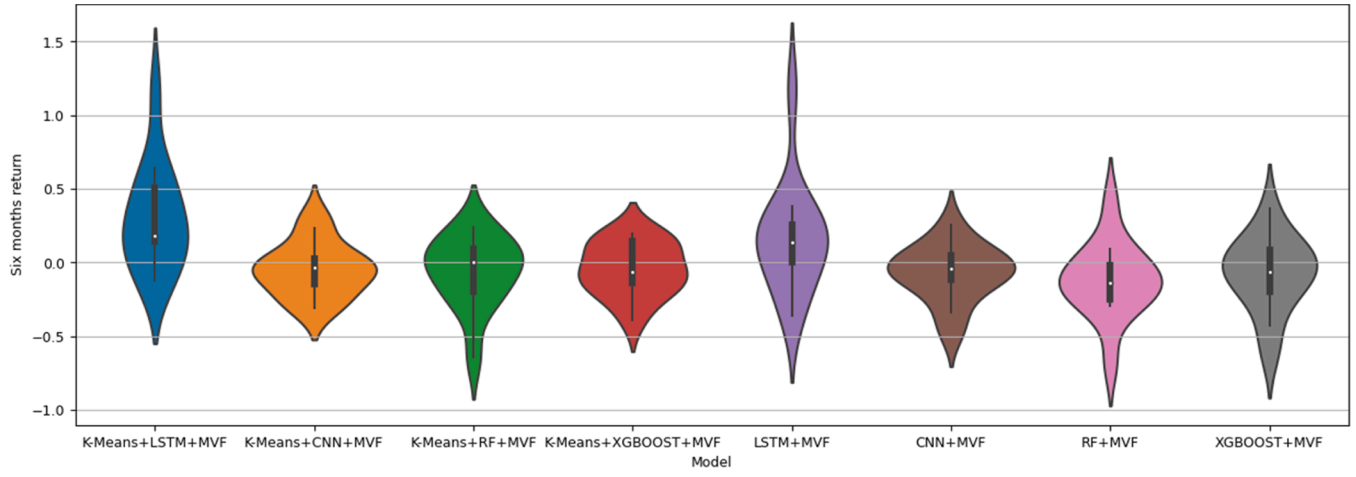

Fig. 13. Violin plot of daily returns, including transaction cost (20bps).

not only reduces computational complexity by eliminating the need for separate forecasting models for each stock but also enhances forecasting accuracy and daily trading portfolio construction performance.

The proposed models in the current study have been tested and implemented on a sample of shares from a large market on the US stock exchange, making them applicable to other markets as well. In this study, a limited number of shares were used for simplicity in calculations and to demonstrate the superiority of the proposed model. Future studies can involve the development of the proposed models for a more extensive set of assets. Future research could also examine other

clustering algorithms, such as hierarchical clustering or Gaussian Mixture Models. Furthermore, incorporating feature selection models to identify efficient features is another possible direction that can be explored in future research.

CRediT authorship contribution statement

Mahdi Ashrafzadeh: Software, Formal analysis, Conceptualization, Writing – review & editing. **Mohammad Sadrani:** Writing – review & editing, Validation, Methodology, Investigation, Conceptualization, Formal analysis. **Sarfaraz Hashemkhani Zolfani:** Writing – review & editing, Validation, Investigation, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Appendix A. Details of the input features.

Table A1.

Table A.1
Description of the selected input features.

Indicator	Formula	Assumptions
Simple Moving Average	$SMA_T = \frac{C_{n-T+1} + C_{n-T+2} + \ldots + C_n}{T} = \frac{1}{T}\sum_{i=n-T+1}^{n} C_i$	C_i = *Close price at time i* T = *The window or total periods* *T=14*
Moving average convergence/ divergence	$EMA_t = \left(Value_t \times \left(\frac{Smoothing}{1+Days}\right)\right) + EMA_{t-1} \times \left(1 - \left(\frac{Smoothing}{1+Days}\right)\right)$ $MACD = EMA_A - EMA_B$	*Smoothing = 2* *A=12* *B=26*
Percentage Price Oscillator	$PPO = \frac{EMA_a - EMA_b}{EMA_b} \times 100$	*EMA = Exponential moving average* EMA_a *:12 period* EMA_b *:26 period*
Stochastic Oscillator	$\%K = \left(\frac{C - L_T}{H_T - L_T}\right) \times 100$ $\%D = SMA(\%K)_N$	C = *The most recent closing price* L_T = *The lowest price over the last T periods* H_T = *The highest price over the last T periods* $\%K$ = *The current value of the stochastic indicator* $\%D$ = *N-period simple moving average of %K* *T=14* *N=3*
Relative Strength Index	$gain_t = C_t - C_{t-1}$ $loss_t = C_{t-1} - C_t$ $RS = \frac{SMA(gain)_T}{SMA(loss)_T}$ $RSI = 100 - \left[\frac{100}{1+RS}\right]$	C_t = Close price at time t *SMA (gain)*$_T$: T-period simple moving average of gain *SMA (loss)*$_T$: T-period simple moving average of loss T=14
Average True Range	$TR_t = Max[(H_t - L_t), \lvert H_t - C_{t-1}\rvert, \lvert L_t - C_{t-1}\rvert]$ $ATR = SMA(TR)_T$	H_t = The highest price at time t L_t = The lowest price at time t C_t = Close price at time t TR_t = True range at time t ATR= T-period simple moving average of TR T=14
Lagged variables	$Lag_1 = ln\left(\frac{C_t}{C_{t-1}}\right)$ $Lag_2 = ln\left(\frac{C_{t-1}}{C_{t-2}}\right)$ $Lag_3 = ln\left(\frac{C_{t-2}}{C_{t-3}}\right)$ $Lag_4 = ln\left(\frac{C_{t-3}}{C_{t-4}}\right)$	C_t = Close price at time t

Data availability

Data will be made available on request.

References

[1] T. Fischer, C. Krauss, Deep learning with long short-term memory networks for financial market predictions, Eur. J. Oper. Res. 270 (2018) 654–669.
[2] C. Krauss, X.A. Do, N. Huck, Deep neural networks, gradient-boosted trees, random forests: Statistical arbitrage on the S & P 500, Eur. J. Oper. Res. 259 (2017) 689–702.
[3] S.I. Lee, S.J. Yoo, Threshold-based portfolio: the role of the threshold and its applications, J. Supercomput. 76 (10) (2020) 8040–8057.
[4] H. Markowitz, Portfolio selection, J. Finance 7 (1) (1952) 77–91.
[5] M.C. Jensen, Some anomalous evidence regarding market efficiency, J. Financ. Econ. 6 (2/3) (1978) 95–101.
[6] F.J. Fabozzi, F. Gupta, H.M. Markowitz, The legacy of modern portfolio theory, J. Invest. 11 (2002) 7–22.
[7] M. Ballings, D.V.D. Poel, N. Hespeels, R. Gryp, Evaluating multiple classifiers for stock price direction prediction, Expert Syst. Appl. 42 (2015) 7046–7056.
[8] A. Booth, E. Gerding, F. Mcgroarty, Automated trading with performance weighted random forests and seasonality, Expert Syst. Appl. 41 (2014) 3651–3661.
[9] Y. Ma, R. Han, W. Wang, Portfolio optimization with return prediction using deep learning and machine learning, Expert Syst. Appl. 165 (2021) 113973.
[10] J. Patel, S. Shah, P. Thakkar, Predicting stock and stock price index movement using Trend Deterministic Data Preparation and machine learning techniques, Expert Syst. Appl. 42 (2015) 259–268.

[11] J. Nobre, R.F. Neves, Combining principal component analysis, discrete wavelet transform and XGBoost to trade in the financial markets, Expert Syst. Appl. 125 (2019) 181–194.
[12] W. Chen, H. Zhang, M.K. Mehlawat, L. Jia, Mean–variance portfolio optimization using machine learning-based stock price prediction, Appl. Soft Comput. 100 (2021) 106943.
[13] M. Ashrafzadeh, H.M. Taheri, M. Gharehgozlou, S. Hashemkhani Zolfani, Clustering-based return prediction model for stock pre-selection in portfolio optimization using PSO-CNN+MVF, J. King Saud Univ. - Comput. Inf. Sci. 35 (9) (2023) 101737.
[14] Z. Hu, Y. Zhao, M. Khushi, A survey of forex and stock price prediction using deep learning, Appl. Syst. Innov. 4 (1) (2021) 9.
[15] N. Oliveira, P. Cortez, N. Areal, The impact of microblogging data for stock market prediction: Using Twitter to predict returns, volatility, trading volume and survey sentiment indices, Expert Syst. Appl. 73 (2017) 125–144.
[16] O.B. Sezer, A.M. Ozbayoglu, Algorithmic Financial Trading with deep Convolutional Neural Networks: Time Series to image Conversion approach, Appl. Soft Comput. 70 (2018) 525–538.
[17] Y.J. Chen, Y.J. Hao, Integrating principle component analysis and weighted support vector machine for stock trading signals prediction, Neurocomputing 321 (2018) 381–402.
[18] G.J. Deboeck, Trading on the edge: Neural, genetic, and fuzzy systems for chaotic financial markets, Wiley, New York, 1994.
[19] F.D.A Paiva, R.T.N. Cardoso, G.P. Hanaoka, W.M Duarte, Decision– making for financial trading: A fusion approach of machine learning and portfolio selection, Expert Syst. Appl. 115 (2019) 635–655.
[20] G.E. Box, G.M. Jenkins, G.C. Reinsel, G.M. Ljung, Time series analysis: forecasting and control, John Wiley & Sons, 2015.
[21] A.A. Adebiyi, A.O. Adewumi, C.K. Ayo, Comparison of ARIMA and artificial neural networks models for stock price prediction, J. Appl. Math. 2014 (2014) 614342.
[22] J.V. Hansen, R.D. Nelson, Data mining of time series using stacked generalizers, Neurocomputing 43 (2002) 173–184.
[23] A.W. Li, G.S. Bastos, Stock market forecasting using deep learning and technical analysis: a systematic review, IEEE Access 8 (2020) 185232–185242.
[24] W. Wang, W. Li, N. Zhang, K. Liu, Portfolio formation with preselection using deep learning from long-term financial data, Expert Syst. Appl. 143 (2020) 113042.
[25] F.D. Freitas, A.F. De Souza, A.R. De Almeida, Prediction-based portfolio optimization model using neural networks, Neurocomputing 72 (10-12) (2009) 2155–2170.
[26] C.Y. Hao, J.Q. Wang, W. Xu, Prediction-based portfolio selection model using support vector machines, in: Proceedings of sixth international conference on business intelligence and financial engineering, 2013, pp. 567–571.
[27] P.N. Kolm, R. Tütüncü, F.J. Fabozzi, 60 Years of portfolio optimisation: Practical challenges and current trends, Eur. J. Oper. Res. 234 (2014) 356–371.
[28] J.R. Yu, W.J.P. Chiou, W.Y. Lee, S.J. Lin, Portfolio models with return forecasting and transaction costs, Int. Rev. Econ. Finance 66 (2020) 118–130.
[29] F.G. Ferreira, A.H. Gandomi, R.T. Cardoso, Artificial intelligence applied to stock market trading: a review, IEEE Access 9 (2021) 30898–30917.
[30] N. Nazareth, Y.V. Ramana Reddy, Financial applications of machine learning: A literature review, Expert Syst. Appl. 219 (2023) 119640.
[31] J. Lee, B. Xia, Analyzing the dynamics between crude oil spot prices and futures prices by maturity terms: Deep learning approaches to futures-based forecasting, Results Eng. 24 (2024) 103086, https://doi.org/10.1016/j.rineng.2024.103086.
[32] S.J. Deng, X.Y. Min, Applied optimization in Global Efficient Portfolio Construction using Earning Forecasts, J. Invest. 22 (2013) 104–114.
[33] W. Chen, H. Zhang, L. Jia, A novel two-stage method for well-diversified portfolio construction based on stock return prediction using machine learning, N. Am. J. Econ. Finance 63 (2022) 101818.
[34] J. Behera, A.K. Pasayat, H. Behera, P. Kumar, Prediction based mean-value-at-risk portfolio optimization using machine learning regression algorithms for multi-national stock markets, Eng. Appl. Artif. Intell. 120 (2023) 105843.
[35] H. Yun, M. Lee, Y.S. Kang, J. Seok, Portfolio management via two-stage deep learning with a joint cost, Expert Syst. Appl. 143 (2020) 113041.
[36] J. Kim, M. Lee, Portfolio optimization using predictive auxiliary classifier generative adversarial networks, Eng. Appl. Artif. Intell. 125 (2023) 106739.
[37] E. Hoseinzade, S. Haratizadeh, CNNpred: CNN-based stock market prediction using a diverse set of variables, Expert Syst. Appl. 129 (2019) 273–285.
[38] S. Abolmakarem, F. Abdi, K. Khalili-Damghani, H. Didehkhani, Predictive multi-period multi-objective portfolio optimization based on higher order moments: Deep learning approach, Comput. Ind. Eng. 183 (2023) 109450.
[39] Y. Ma, W. Wang, Q. Ma, A novel prediction based portfolio optimization model using deep learning, Comput. Ind. Eng. 177 (2023) 109023.
[40] S. Almahdi, S.Y. Yang, An adaptive portfolio trading system: A risk-return portfolio optimization using recurrent reinforcement learning with expected maximum drawdown, Expert Syst. Appl. 87 (2017) 267–279.
[41] S. Almahdi, S.Y. Yang, A constrained portfolio trading system using particle swarm algorithm and recurrent reinforcement learning, Expert Syst. Appl. 130 (2019) 145–156.
[42] H. Park, M.K. Sim, D.G. Choi, An intelligent financial portfolio trading strategy using deep Q-learning, Expert Syst. Appl. 158 (2020) 113573.
[43] J.V. Sáenz, F.M. Quiroga, A.F. Bariviera, Data vs. information: Using clustering techniques to enhance stock returns forecasting, Int. Rev. Financ. Anal. 88 (2023) 102657.
[44] A. Sherly, J.V.E. Mary Subaja Christo, A hybrid approach to time series forecasting: Integrating ARIMA and prophet for improved accuracy, Results Eng. 27 (2025) 105703.
[45] M. Li, Y. Zhu, Y. Shen, M. Angelova, Clustering-enhanced stock price prediction using deep learning, World Wide Web 26 (1) (2023) 207–232.
[46] F. Zhou, Q. Zhang, D. Sornette, L. Jiang, Cascading logistic regression onto gradient boosted decision trees for forecasting and trading stock indices, Appl. Soft Comput. 84 (2019) 105747.
[47] R. Bellman, R. Kalaba, On adaptive control processes, IRE Trans. Autom. Control 4 (2) (1959) 1–9.
[48] J. Gu, X. Jin, A simple approximation for dynamic time warping search in large time series database, in: Intelligent Data Engineering and Automated Learning–IDEAL 2006: 7th International Conference, Springer Berlin Heidelberg, Burgos, Spain, 2006, pp. 841–848. September 20-232006. Proceedings 7.
[49] V. Niennattrakul, C.A. Ratanamahatana, On Clustering Multimedia Time Series Data Using K-Means and Dynamic Time Warping, in: 2007 International Conference on Multimedia and Ubiquitous Engineering, 2007, pp. 733–738.
[50] T.M. Kodinariya, P.R. Makwana, Review on determining number of Cluster in K-Means Clustering, Int. J. 1 (6) (2013) 90–95.
[51] E. Forgy, Cluster Analysis of Multivariate Data: Efficiency versus Interpretability of Classifications, Biometrics 21 (1965) 768–780.
[52] J. MacQueen, Some methods for classification and analysis of multivariate observations, 1967.
[53] M. Ester, H.P. Kriegel, J. Sander, X. Xu, A density-based algorithm for discovering clusters in large spatial databases with noise, In kdd 96 (34) (1996) 226–231.
[54] S. Hochreiter, J. Schmidhuber, Long short-term memory, Neural Comput. 9 (8) (1997) 1735–1780.
[55] Y. LeCun, Y. Bengio, Convolutional networks for images, speech, and time series, Handb. Brain Theory Neural Netw. 3361 (10) (1995) 1995.
[56] S. Mehtab, J. Sen, Stock price prediction using CNN and LSTM-based deep learning models, in: 2020 International Conference on Decision Aid Sciences and Application (DASA), IEEE, 2020, pp. 447–453.
[57] S. Selvin, R. Vinayakumar, E.A. Gopalakrishnan, V.K. Menon, K.P. Soman, Stock price prediction using LSTM, RNN and CNN-sliding window model, in: 2017 international conference on advances in computing, communications and informatics (icacci), IEEE, 2017, pp. 1643–1647.
[58] T.K. Ho, Random decision forests, in: Proceedings of 3rd international conference on document analysis and recognition 1, IEEE, 1995, pp. 278–282.
[59] I.K. Nti, A.F. Adekoya, B.A. Weyori, A comprehensive evaluation of ensemble learning for stock-market prediction, J. Big Data 7 (1) (2020) 1–40.
[60] J. Patel, S. Shah, P. Thakkar, K. Kotecha, Predicting stock market index using fusion of machine learning techniques, Expert Syst. Appl. 42 (4) (2015) 2162–2172.
[61] M. Vijh, D. Chandola, V.A. Tikkiwal, A. Kumar, Stock Closing Price Prediction using Machine Learning Techniques, Procedia Comput. Sci. 167 (2020) 599–606.
[62] T. Chen, C. Guestrin, Xgboost: A scalable tree boosting system, in: Proceedings of the 22nd acm sigkdd international conference on knowledge discovery and data mining, 2016, pp. 785–794.
[63] Y. Zhang, Stock Price Prediction Method Based on XGboost Algorithm, in: Proceedings of the 2022 International Conference on Bigdata Blockchain and Economy Management (ICBBEM 2022), Atlantis Press, 2022, pp. 595–603. ISSN: 2589-4919.
[64] D.B. Keim, A. Madhavan, The cost of institutional equity trades, Financ. Anal. J. 54 (1998) 50–69.
[65] M.-A. Mittermayer, Forecasting intraday stock price trends with text mining techniques. In system sciences, in: 2004. proceedings of the 37th annual hawaii international conference on, IEEE, 2004 (pp. 10–pp).